

What is LVM?

In simple terms, **LVM (Logical Volume Manager)** is a tool in Linux that helps you manage your computer's storage. It lets you create, resize, and organize your storage space without being stuck with the size of your hard drives or partitions.

Imagine LVM as a flexible **storage organizer**. You can move, resize, and even combine storage from different hard drives to create one big storage space that's easy to manage. It helps you make the most of your storage and avoid running out of space.

Don't worry if this sounds a bit complicated right now. As you read on, we'll explain how volumes work and how LVM fits into the picture.

What are volumes in linux ?

volumes in Linux are like containers that hold your data. They represent storage space on your computer or server, and you can use them to store files and programs.

These volumes can be physical (like an SSD or HDD), partitioned sections of a physical disk, or even logical/virtual disks managed by a tool called **LVM (Logical Volume Manager)**.

Types of Volumes

1. Physical Volume (PV)

- A **physical volume** is the actual storage device, like an **HDD (Hard Disk Drive)** or **SSD (Solid State Drive)**.

Example: Your laptop's SSD or an external hard drive.

 **Key Point:** Every volume must exist on a physical storage device.

2. Volume Group (VG)

- A storage pool made from multiple physical volumes.

Example: Combining an SSD[10 GB] and HDD [10 GB] in one group .

 **Key Point:** VGs merge multiple devices into one.

3. Logical Volume (LV) - Managed by LVM

- A flexible storage unit inside a volume group, easy to resize or move.

Example: Creating a 10GB Logical Volume (LV) from a 20GB Volume Group (VG)

📌 **Key Point:** LVs allow for dynamic storage management, including resizing and moving.

What is AWS EBS?

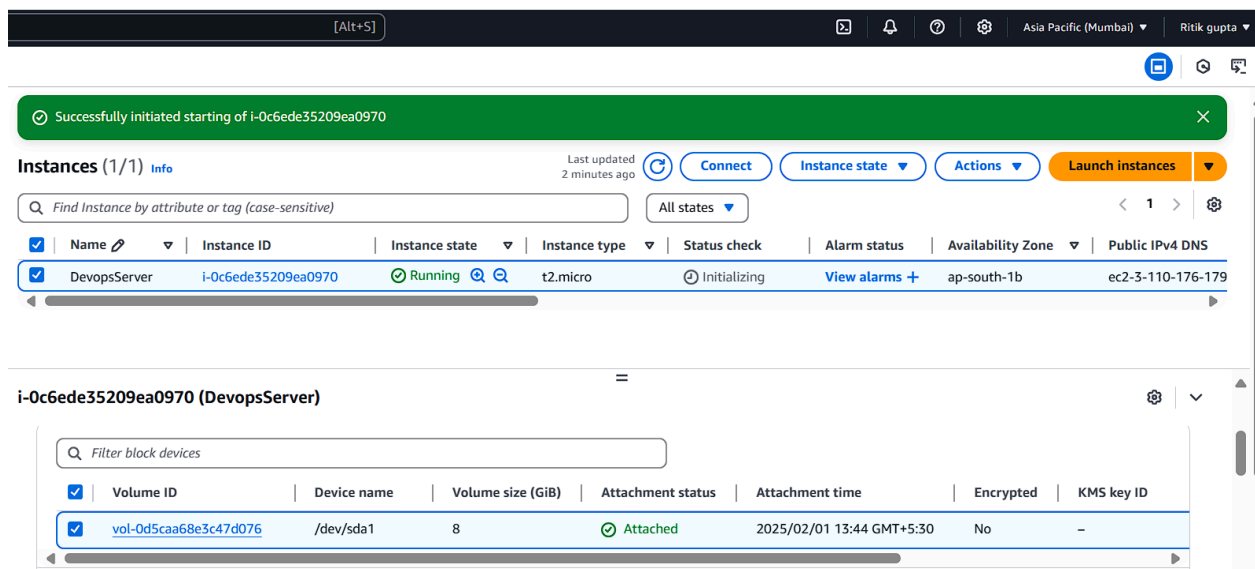
AWS **EBS (Elastic Block Store)** is a block storage service that provides persistent storage for Amazon EC2 instances. It behaves like a physical volume (PV) in Linux, offering block-level storage that can be attached to instances.

📌 **Key Point:** AWS EBS is **not an LV**, but it is **used as a PV in LVM**.

📌 You can create **Logical Volumes (LVs)** inside AWS EBS by using **LVM** for more flexibility in managing your storage.

Now, let's set up LVM on top of AWS EBS

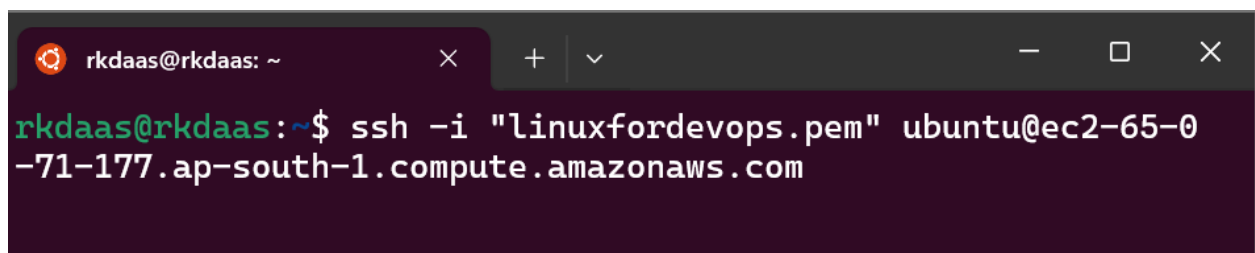
I already have an EC2 instance running, but you can launch one yourself—it's a simple process. By default, AWS provides an **8GB EBS volume** with your instance unless you customize the storage settings during creation.



The screenshot shows the AWS Management Console interface. At the top, a green banner indicates "Successfully initiated starting of i-Oc6ede35209ea0970". Below this, the "Instances (1/1)" section shows a table with one instance, "DevopsServer", in a "Running" state. The instance details show it is a "t2.micro" type in the "ap-south-1b" availability zone. Below the instance details, the "i-Oc6ede35209ea0970 (DevopsServer)" section shows a table of attached block devices. The table has columns for Volume ID, Device name, Volume size (GiB), Attachment status, Attachment time, Encrypted, and KMS key ID. One volume is listed: "vol-0d5caa68e3c47d076" attached to "/dev/sda1" with a size of 8 GiB and an "Attached" status.

Volume ID	Device name	Volume size (GiB)	Attachment status	Attachment time	Encrypted	KMS key ID
vol-0d5caa68e3c47d076	/dev/sda1	8	Attached	2025/02/01 13:44 GMT+5:30	No	-

Connect to Remote Server via SSH



The screenshot shows a terminal window with the prompt "rkdaas@rkdaas: ~". The user has entered the command "ssh -i 'linuxfordevops.pem' ubuntu@ec2-65-0-71-177.ap-south-1.compute.amazonaws.com".

```
rkdaas@rkdaas:~$ ssh -i "linuxfordevops.pem" ubuntu@ec2-65-0-71-177.ap-south-1.compute.amazonaws.com
```

Now Let's understand some Important commands

1- Lsblk -This command in Linux shows a clear list of all storage devices and their partitions. Think of it as a map of your system's disks—it tells you what storage is available, how it's divided, and where it's mounted.

```
rkdaas@rkdaas: ~  
ubuntu@ip-172-31-5-208:~$ lsblk  
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS  
loop0        7:0      0 26.3M  1 loop /snap/amazon-ssm-agent/988  
1  
loop1        7:1      0 73.9M  1 loop /snap/core22/1722  
loop2        7:2      0 44.4M  1 loop /snap/snapd/23545  
loop3        7:3      0 73.9M  1 loop /snap/core22/1748  
xvda         202:0     0   8G   0 disk  
├─xvda1      202:1     0   7G   0 part /  
├─xvda14     202:14    0   4M   0 part  
├─xvda15     202:15    0 106M   0 part /boot/efi  
└─xvda16     259:0     0 913M   0 part /boot
```

Breaking it Down

- **NAME** → The disk name (xvda)
- **SIZE** → Total size of the disk
- **TYPE** → Whether it's a disk or partition
- **MOUNT POINT** → If mounted, it shows where (e.g., / or /mnt/data)

- 📌 If a volume is **attached** but **not mounted**, the "MOUNTPOINT" column will be **empty**.
- 📌 If a volume is **mounted**, it will show the directory where it is accessible.

Difference between Attached and mounted

1) Attached:

- The storage (like an EBS volume) is connected to the system.
- It's not yet available for use or accessible by the OS.
- **Key Point:** Just connected, but not ready to store data yet.

2) Mounted:

- The storage is linked to a specific folder (mount point), making it usable.
- You can access it and store data in that folder.

- **Key Point:** Mounted means it's now accessible and ready for use.

Example in AWS:

- When you attach an EBS volume to an EC2 instance, it's connected, but not usable until you mount it to a folder (e.g., /mnt/data).
- **Key Point:** You can't store data on the volume until it's mounted.

2- Df -h - This command helps monitor your filesystem(volumes) space usage and provides a warning when storage is running low, so you can take action before running out of space.

```
ubuntu@ip-172-31-5-208:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        6.8G  1.8G  5.0G  27% /
tmpfs            479M   0    479M   0% /dev/shm
tmpfs            192M  868K  191M   1% /run
tmpfs            5.0M   0    5.0M   0% /run/lock
/dev/xvda16      881M   76M  744M  10% /boot
/dev/xvda15      105M   6.1M   99M   6% /boot/efi
tmpfs            96M   12K   96M   1% /run/user/1000
```

 **Key Point:** You can see i have used filesystem and volumes interchangeably in this context

Here's why:

- ① **Volume** refers to a logical storage unit, such as a disk or partition.
- ② **File System** refers to the way data is organized and stored on that volume.

So, while "filesystem" technically refers to how data is organized on a volume, it's common to use the term "filesystem" when talking about volumes in this context, because that's what the df-h command reports on.

Let's roll out some additional EBS volumes and attach to our EC2 instance

I manually created volumes of sizes 10GB, 12GB, and 14GB, while the 8GB volume was automatically created when I launched the EC2 instance.

Successfully created volume vol-0b09284384f75fc37.

Volumes (4) Info

Actions Create volume

Search

	Name	Volume ID	Type	Size	IOPS	Throughput	Snapshot ID	Created	Availability
☐	-	vol-0b09284384f75fc37	gp3	14 GiB	3000	125	-	2025/02/04 22:19 GMT+5:...	ap-south-
☐	-	vol-0d5caa68e3c47d076	gp3	8 GiB	3000	125	snap-08533e3...	2025/02/01 13:44 GMT+5:...	ap-south-
☐	-	vol-03c854d1306f72093	gp3	10 GiB	3000	125	-	2025/02/04 22:18 GMT+5:...	ap-south-
☐	-	vol-0a8f7a6b2e0cc0104	gp3	12 GiB	3000	125	-	2025/02/04 22:18 GMT+5:...	ap-south-

Now that we've created the volumes, we can proceed to attach them to our EC2 instance.

aws Search [Alt+S]

EC2 > Volumes > vol-0b09284384f75fc37 > Attach volume

Basic details

Volume ID
vol-0b09284384f75fc37

Availability Zone
ap-south-1b

Instance Info
i-0c6ede35209ea0970
(DevopsServer) (running)

Only instances in the same Availability Zone as the selected volume are displayed.

Device name Info
/dev/sdf

Recommended device names for Linux: /dev/sda1 for root volume. /dev/sd[f-p] for data volumes.

ⓘ Newer Linux kernels may rename your devices to /dev/xvdf through /dev/xvdp internally, even when the device name entered here (and shown in the details) is /dev/sdf through /dev/sdp.

📌 **Key Point:** Ensure you use the correct device name as per the recommended naming conventions. I've named my 10 GB EBS volume as /dev/sdf, but Linux may automatically rename it to /dev/xvdf as mentioned .

Similarly, I've attached all the volumes, and you can do the same.

Successfully attached volume [vol-0a8f7a6b2e0cc0104](#) to instance [i-0c6ede35209ea0970](#).

Volumes (4) Info

Saved filter sets
Choose filter set ▾

Q Search

< 1 > ⚙

Throughput ▾	Snapshot ID ▾	Created ▾	Availability Zone ▾	Volume state ▾	Alarm status	Attached resources ▾
125	-	2025/02/04 22:19 GMT+5:...	ap-south-1b	✔ In-use	No alarms	+ i-0c6ede35209ea0970 (De...
125	snap-08533e3...	2025/02/01 13:44 GMT+5:...	ap-south-1b	✔ In-use	No alarms	+ i-0c6ede35209ea0970 (De...
125	-	2025/02/04 22:18 GMT+5:...	ap-south-1b	✔ In-use	No alarms	+ i-0c6ede35209ea0970 (De...
125	-	2025/02/04 22:18 GMT+5:...	ap-south-1b	✔ In-use	No alarms	+ i-0c6ede35209ea0970 (De...

Now that we've successfully attached the volumes

let's check if they are showing up on our EC2 instance.

```
ubuntu@ip-172-31-5-208:~$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0        7:0    0 26.3M  1 loop /snap/amazon-ssm-agent/988
1
loop1        7:1    0 73.9M  1 loop /snap/core22/1722
loop2        7:2    0 44.4M  1 loop /snap/snapd/23545
loop3        7:3    0 73.9M  1 loop /snap/core22/1748
xvda        202:0    0    8G    0 disk
├─xvda1      202:1    0    7G    0 part /
├─xvda14     202:14   0    4M    0 part
├─xvda15     202:15   0   106M  0 part /boot/efi
└─xvda16     259:0    0   913M  0 part /boot
xvdf        202:80   0   14G    0 disk
xvdg        202:96   0   10G    0 disk
xvdh        202:112  0   12G    0 disk
```

We previously added new volumes and named them **sdf**, **sdg**, and **sdh**. However, Linux renamed them to **xvdf**, **xvdg**, and **xvdh**. While the volumes are present as we can see, their mount points are empty because they have only been attached, not mounted yet.

Before Moving further lets go through some lvm commands

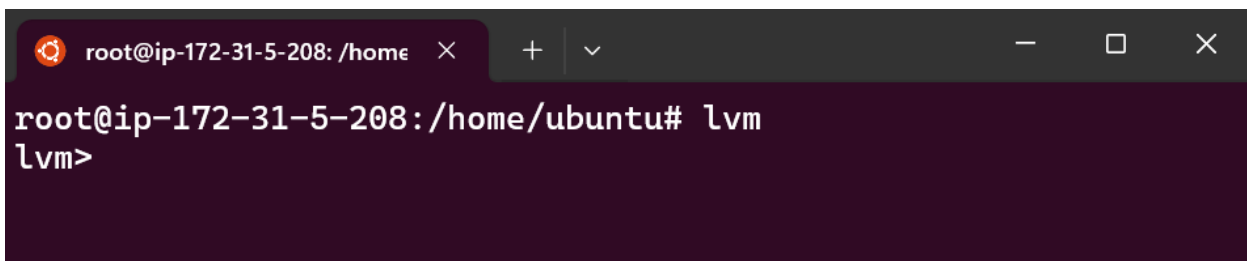
```
pvcreate - Create a physical volume (PV)
vgcreate - Create a volume group (VG)
lvcreate - Create a logical volume (LV)
lvextend - Extend LV size
lvreduce - Reduce LV size
vgextend - Add PV to VG
vgreduce - Remove PV from VG
```

```
lvremove - Remove LV
vgremove - Remove VG
pvremove - Remove PV
pvs      - gives summary of all physical volumes
vgs      - gives summary of all volume groups
lvs      - gives summary of all logical volumes
```

Become a root user

```
ubuntu@ip-172-31-5-208:~$ sudo su
root@ip-172-31-5-208:/home/ubuntu#
```

Run lvm command

A terminal window with a dark purple background. The title bar shows 'root@ip-172-31-5-208: /home' and standard window controls. The prompt is 'root@ip-172-31-5-208:/home/ubuntu#'. The user has entered 'lvm' and the prompt has changed to 'lvm>'.

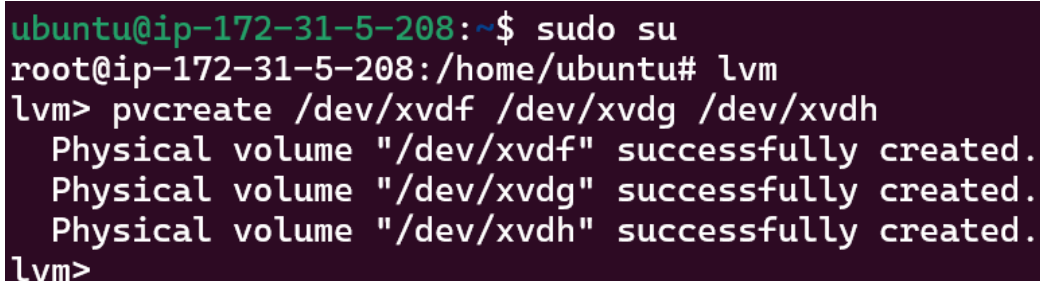
```
root@ip-172-31-5-208:/home/ubuntu# lvm
lvm>
```

Now we are inside the Logical volume manager [LVM]

Let's create the Physical volume

EBS volumes, by default, are just raw devices that the operating system sees. To manage them with LVM (Logical Volume Manager), you need to initialize them as **Physical Volumes (PVs)** using the `pvcreate` command.

So your physical volumes are created -

A terminal window with a dark purple background. The title bar shows 'ubuntu@ip-172-31-5-208: ~\$' and standard window controls. The user has entered 'sudo su' and the prompt is 'root@ip-172-31-5-208:/home/ubuntu#'. The user has entered 'lvm' and the prompt is 'lvm>'. The user has entered 'pvcreate /dev/xvdf /dev/xvdg /dev/xvdh' and the output shows three lines of success messages. The prompt is still 'lvm>'.

```
ubuntu@ip-172-31-5-208:~$ sudo su
root@ip-172-31-5-208:/home/ubuntu# lvm
lvm> pvcreate /dev/xvdf /dev/xvdg /dev/xvdh
Physical volume "/dev/xvdf" successfully created.
Physical volume "/dev/xvdg" successfully created.
Physical volume "/dev/xvdh" successfully created.
lvm>
```

You can see your physical volumes like this -

```
root@ip-172-31-5-208:/home/ubuntu# lvm
lvm> pvs
  PV          VG Fmt  Attr PSize  PFree
  /dev/xvdf   lvm2 ---  14.00g 14.00g
  /dev/xvdg   lvm2 ---  10.00g 10.00g
  /dev/xvdh   lvm2 ---  12.00g 12.00g
lvm>
```

Let's create the volume group

Creating a volume group with /dev/xvdf and /dev/xvdg

```
lvm > vgcreate tws_vg /dev/xvdf /dev/xvdg
Volume group "tws_vg" successfully created
lvm>
```

you can see your volume groups like this -

```
lvm> vgs
  VG      #PV #LV #SN Attr   VSize  VFree
  tws_vg   2   0   0 wz--n- 23.99g 23.99g
lvm>
```

Now finally Let's create the logical volume

Creating a logical volume tws_lv of 10GB .

```
root@ip-172-31-5-208:/home/ubuntu# lvm
lvm> lvcreate -L 10G -n tws_lv tws_vg
Logical volume "tws_lv" created.
lvm>
```


You can see your logical volumes like this -

```
lvm> lvs
  LV      VG      Attr      LSize   Pool Origin Data%  Meta%
Move Log Cpy%Sync Convert
  tws_lv  tws_vg  -wi-a----- 10.00g

lvm>
```

View Attached Storage Devices with lsblk

```
root@ip-172-31-5-208:/home/ubuntu# lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0                               7:0      0 26.3M  1 loop /snap/amazon-ssm-agent/9881
loop1                               7:1      0 73.9M  1 loop /snap/core22/1722
loop2                               7:2      0 44.4M  1 loop /snap/snapd/23545
loop3                               7:3      0 73.9M  1 loop /snap/core22/1748
xvda                                202:0     0    8G   0 disk
├─xvda1                             202:1     0    7G   0 part /
├─xvda14                            202:14    0    4M   0 part
├─xvda15                            202:15    0 106M   0 part /boot/efi
└─xvda16                            259:0     0 913M   0 part /boot
xvdf                                202:80    0   14G   0 disk
└─tws_vg-tws_lv                    252:0     0   10G   0 lvm
xvdg                                202:96    0   10G   0 disk
xvdh                                202:112   0   12G   0 disk
root@ip-172-31-5-208:/home/ubuntu#
```

 **Key Point:** We can see that a **10GB logical volume** has been created, which is currently allocated from **xvdf**. In the future, if we create more logical volumes, they might be allocated from **xvdg** as well.

Check Disk Space Usage with df -h

```
root@ip-172-31-5-208:/home/ubuntu# df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        6.8G  1.8G  4.9G  27% /
tmpfs            479M   0    479M   0% /dev/shm
tmpfs            192M  884K  191M   1% /run
tmpfs            5.0M   0    5.0M   0% /run/lock
/dev/xvda16      881M   76M  744M  10% /boot
/dev/xvda15      105M   6.1M   99M   6% /boot/efi
tmpfs            96M   12K   96M   1% /run/user/1000
root@ip-172-31-5-208:/home/ubuntu#
```

🔑 **Key Point** :But surprisingly We can't see **xvdf**, **xvdg**, and **xvdh** because they are not mounted yet. Let's go ahead and mount them.

Let's do Mounting Now

1- Mounting Logical volume tws_lv

- **Create a Mount Point**

Before mounting, we need to create a directory where the logical volume will be accessible.

```
root@ip-172-31-5-208:/home/ubuntu# mkdir /mnt/tws_lv_mount
```

- **Format the Logical Volume**

To create a filesystem on the logical volume, we use the **mkfs.ext4** command.

You can format any logical volume by specifying its correct path, which follows the format:

```
syntax - mkfs.ext4 /dev/<VG_NAME>/<LV_NAME>
```

Replace **<VG_NAME>** and **<LV_NAME>** with your actual volume group and logical volume names.

```
root@ip-172-31-5-208: /home × + ∨ − □ ×
root@ip-172-31-5-208:/home/ubuntu# mkfs.ext4 /dev/tws_vg/tws_lv
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 2621440 4k blocks and 655360 inodes
Filesystem UUID: c4de9e18-1d44-4d51-94c3-3ff430e88e68
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736
, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information:
done
```

- **Mount the logical volume**

```
mount <logical_volume> <mount_point>
<logical_volume>: Path to the logical volume (e.g., /dev/tws_vg/tws_lv).
<mount_point>: Directory where the volume will be mounted (e.g.,
/mnt/tws_lv_mount).
```

```
root@ip-172-31-5-208:/home/ubuntu# mount /dev/tws_vg/tws_lv
/mnt/tws_lv_mount
root@ip-172-31-5-208:/home/ubuntu#
```

After running this command, the contents of **/dev/tws_vg/tws_lv** will be available inside **/mnt/tws_lv_mount**.

📌 **Key Point** : To confirm that the logical volume has been successfully mounted, use the **df -h** command:

```
root@ip-172-31-5-208: /home × + ∨ − □ ×
root@ip-172-31-5-208:/home/ubuntu# df -h
Filesystem                Size      Used Avail Use% Mounted on
/dev/root                  6.8G      1.8G   4.9G  27% /
tmpfs                     479M          0  479M   0% /dev/shm
tmpfs                     192M      884K   191M   1% /run
tmpfs                     5.0M          0   5.0M   0% /run/lock
/dev/xvda16                881M       76M   744M  10% /boot
/dev/xvda15               105M       6.1M    99M   6% /boot/efi
tmpfs                     96M       12K    96M   1% /run/user/1000
/dev/mapper/tws_vg-tws_lv  9.8G       24K   9.3G   1% /mnt/tws_lv_mount
root@ip-172-31-5-208:/home/ubuntu#
```

We can see our logical volume **tws_lv** successfully mounted at **/mnt/tws_lv_mount**

Now that the logical volume is mounted, the space is ready for use.

You can create new folders and files within it as needed.

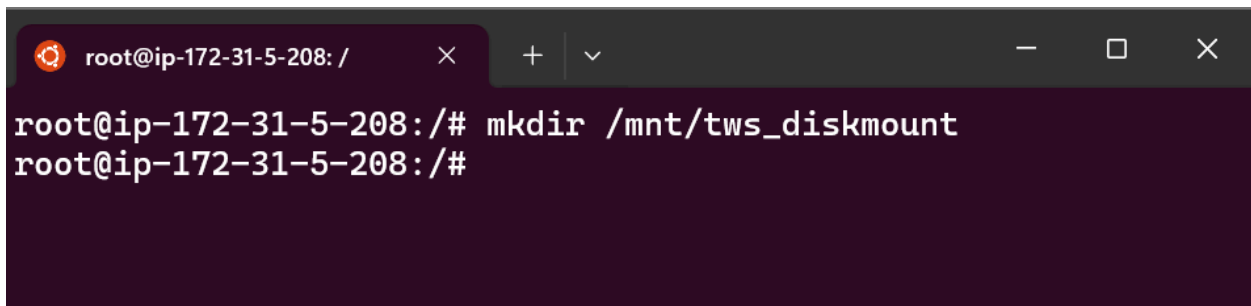
```
root@ip-172-31-5-208:/home/ubuntu# cd /mnt/tws_lv_mount/
root@ip-172-31-5-208:/mnt/tws_lv_mount# mkdir newfolder
root@ip-172-31-5-208:/mnt/tws_lv_mount# touch newfile.txt
root@ip-172-31-5-208:/mnt/tws_lv_mount#
```

```
root@ip-172-31-5-208: /mnt/t × + ∨ − □ ×
root@ip-172-31-5-208:/mnt/tws_lv_mount# ls
lost+found  newfile.txt  newfolder
root@ip-172-31-5-208:/mnt/tws_lv_mount#
```

2- Mounting AWS EBS volume disk directly

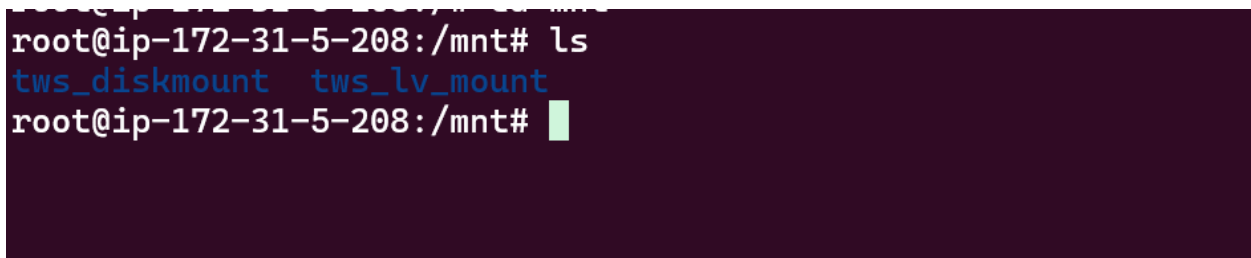
- **Create a Mount Point**

To mount an AWS EBS volume to an EC2 instance, first, you need to create a mount point directory where the volume will be attached.

A terminal window with a dark purple background. The title bar shows 'root@ip-172-31-5-208: /' and window control buttons. The terminal shows the command 'mkdir /mnt/tws_diskmount' being executed successfully, with the prompt returning to 'root@ip-172-31-5-208: /#'.

```
root@ip-172-31-5-208: /# mkdir /mnt/tws_diskmount
root@ip-172-31-5-208: /#
```

The first directory is our mounting point

A terminal window with a dark purple background. The title bar shows 'root@ip-172-31-5-208: /mnt'. The terminal shows the command 'ls' being executed, displaying two directories: 'tws_diskmount' and 'tws_lv_mount'.

```
root@ip-172-31-5-208: /mnt# ls
tws_diskmount  tws_lv_mount
root@ip-172-31-5-208: /mnt#
```

- **Format the AWS EBS volume disk**

To format an AWS EBS volume disk , you can use the following syntax:

```
syntax - mkfs -t ext4 <path_to_aws_ebs_volume>
```

```
root@ip-172-31-5-208:/mnt# mkfs -t ext4 /dev/xvdh
mke2fs 1.47.0 (5-Feb-2023)
/dev/xvdh contains a LVM2_member file system
Proceed anyway? (y,N) y
Creating filesystem with 3145728 4k blocks and 786432 inodes
Filesystem UUID: eaf90f5c-d132-4aa9-b783-2da61b20d57b
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736
, 1605632, 2654208

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information:
done

root@ip-172-31-5-208:/mnt#
```

- **Mount the AWS EBS volume disk**

Use the following command to mount the EBS volume (**/dev/xvdh**) to the mount point (**/mnt/ebs_volume**)

```
root@ip-172-31-5-208: /  × + ▾ - □ ×
root@ip-172-31-5-208:/# mount /dev/xvdh /mnt/tws_diskmount
root@ip-172-31-5-208:/#
```

📌 **Key Point** : To confirm that the logical volume has been successfully mounted, use the **df -h** command:

```

root@ip-172-31-5-208:/# df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        6.8G  1.8G  4.9G  27% /
tmpfs            479M   0  479M   0% /dev/shm
tmpfs            192M  884K  191M   1% /run
tmpfs            5.0M   0   5.0M   0% /run/lock
/dev/xvda16      881M   76M  744M  10% /boot
/dev/xvda15      105M   6.1M   99M   6% /boot/efi
tmpfs            96M   12K   96M   1% /run/user/1000
/dev/mapper/tws_vg-tws_lv 9.8G   28K  9.3G   1% /mnt/tws_lv_mount
/dev/xvdh        12G    24K   12G   1% /mnt/tws_diskmount
root@ip-172-31-5-208:/#

```

You can see that all mounted filesystems are listed, and **/dev/xvdh** (the AWS EBS volume) is mounted at **/mnt/tws_diskmount**. Now, you can make use of this space for storing files and directories.

Conclusion

I hope this guide helped you understand how to efficiently manage storage on AWS EC2 using LVM. From creating Physical Volumes to formatting and mounting AWS EBS volumes, we've covered it all. With this knowledge, you're now equipped to manage scalable and flexible storage effectively. Keep learning and growing!

Hare krishna 🙏